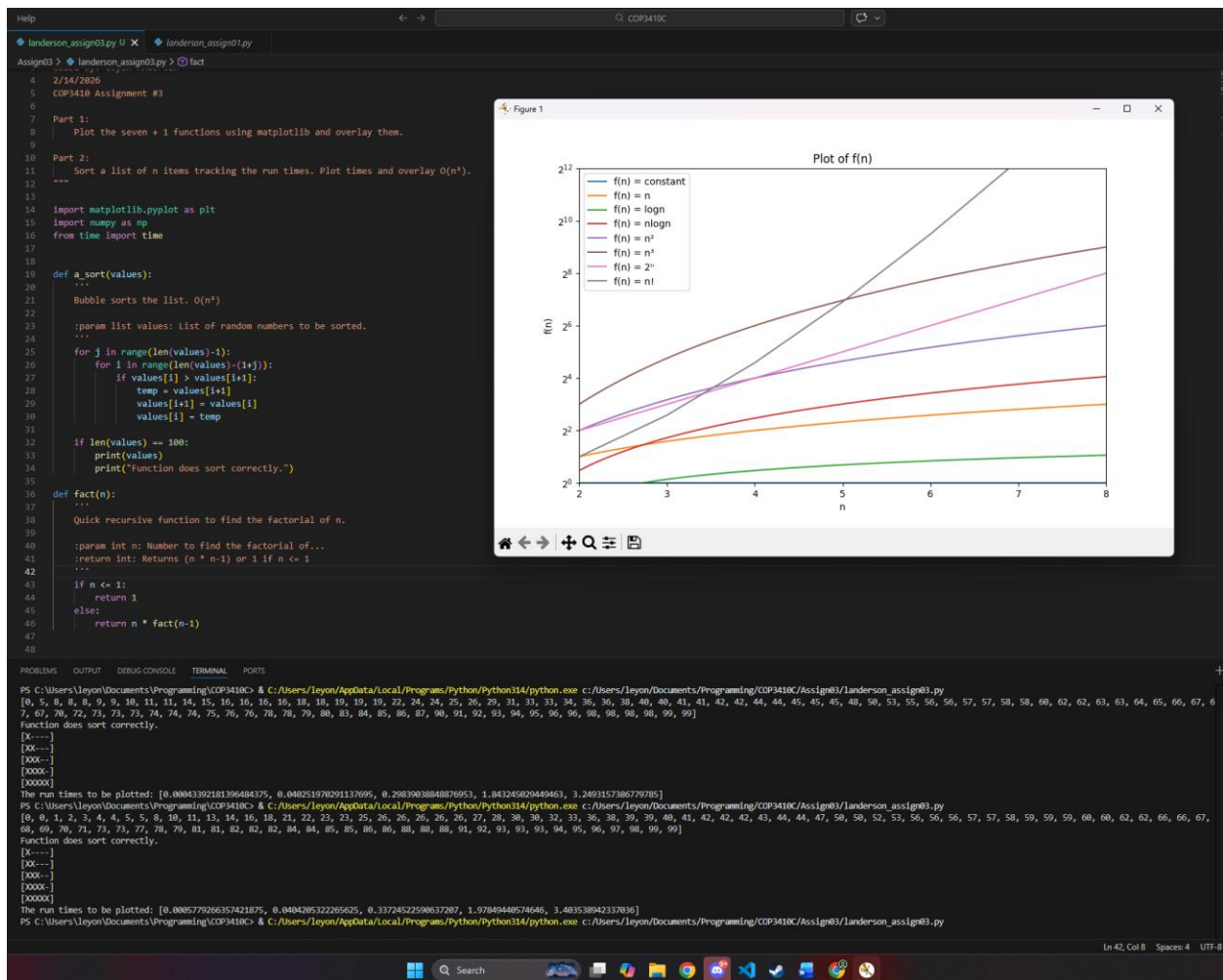
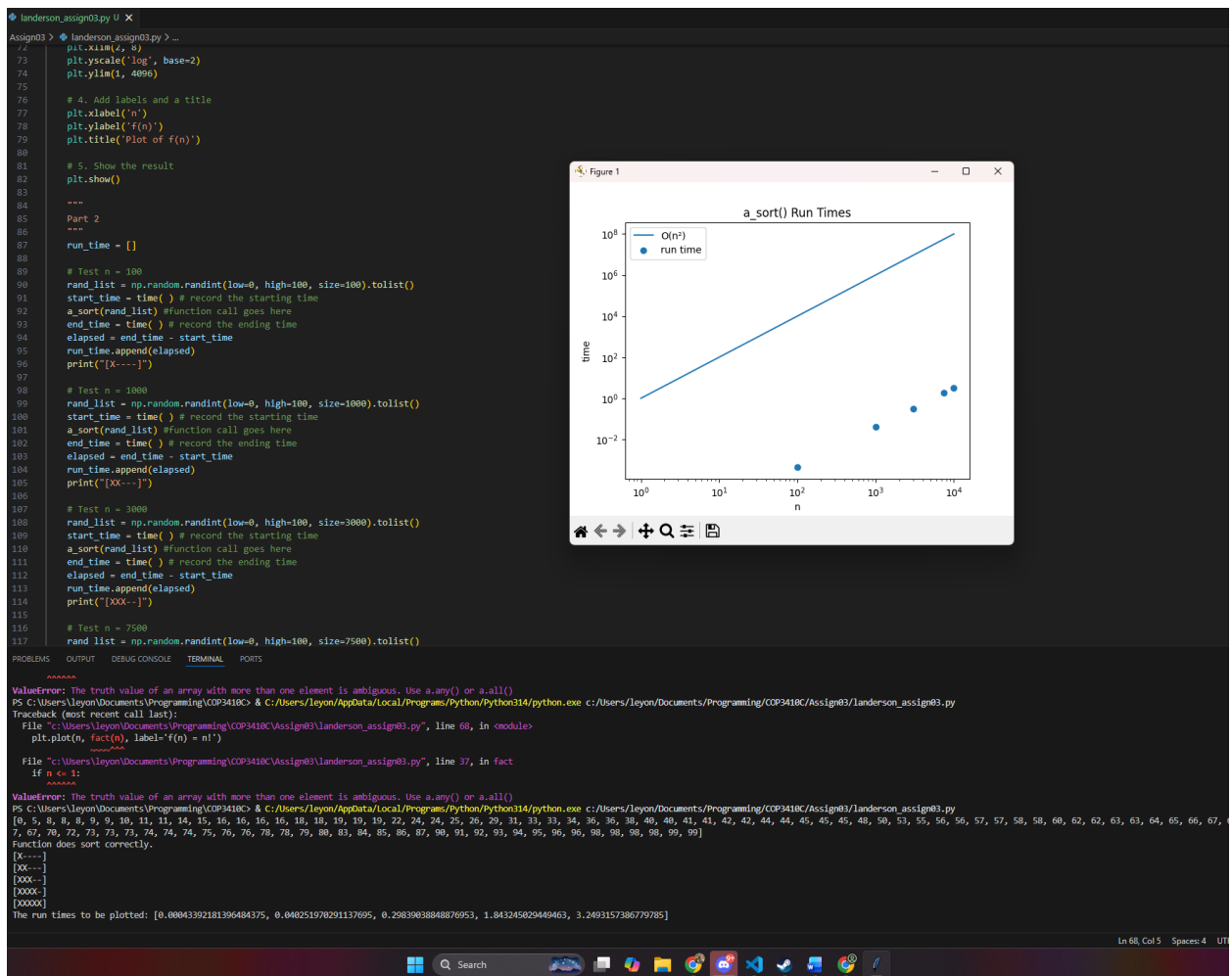




Eight functions overlayed using matplotlib and my own recursive factorial function.



run_time of my sorting function a_sort() is growing at the same rate as $O(n^2)$

```
PS C:\Users\leyon\Documents\Programming\COP3418C> & C:/Users/leyon/AppData/Local/Programs/Python/Python314/python.exe c:/Users/leyon/Documents/Programming/COP3418C/Assign83/landerson_assign83.py
[0, 1, 1, 1, 1, 3, 5, 5, 6, 9, 10, 10, 11, 11, 14, 14, 15, 19, 19, 20, 20, 20, 21, 22, 22, 23, 23, 27, 28, 28, 29, 30, 32, 35, 36, 36, 38, 38, 38, 38, 39, 39, 42, 43, 43, 45, 46, 48, 48, 48, 50, 50, 50, 51, 53, 53, 54, 54, 54, 56, 58, 59, 59, 61, 62, 62, 63, 63, 64, 64, 67, 67, 68, 68, 69, 71, 72, 73, 74, 75, 75, 75, 76, 76, 79, 81, 81, 83, 86, 90, 91, 92, 92, 92, 93, 93, 93, 94, 94, 95, 96, 96, 98, 98]
Function does sort correctly.
[X----]
[XX---]
[XXX--]
[XXXX-]
[XXXXX]
The run times to be plotted: [0.00040221214294433594, 0.03488326072692871, 0.3898489175872883, 1.9347372158421143, 3.48496563911438]
```

The list of 100 is printed at the end of the function to show that a `_sort()` is indeed sorting the list correctly. The list of `run_times` to be plotted are also printed for reference.



```
"""
```

```
Coded by: Leyon Anderson
```

```
2/14/2026
```

```
COP3410 Assignment #3
```

```
Part 1:
```

```
    Plot the seven + 1 functions using matplotlib and overlay them.
```

```
Part 2:
```

```
    Sort a list of n items tracking the run times. Plot times and overlay  $O(n^2)$ .
```

```
"""
```

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
from time import time
```

```
def a_sort(values):
```

```
    '''
```

```
    Bubble sorts the list.  $O(n^2)$ 
```

```
    :param list values: List of random numbers to be sorted.
```

```
    '''
```

```
    for j in range(len(values)-1):
```

```
        for i in range(len(values)-(1+j)):
```

```
            if values[i] > values[i+1]:
```

```
                temp = values[i+1]
```

```
                values[i+1] = values[i]
```

```
                values[i] = temp
```

```
    if len(values) == 100:
```

```
        print(values)
```

```
        print("Function does sort correctly.")
```

```
def fact(n):
```

```
    '''
```

```
    Quick recursive function to find the factorial of n.
```

```
    :param int n: Number to find the factorial of...
```

```
    :return int: Returns (n * n-1) or 1 if n <= 1
```

```
    '''
```

```
    if n <= 1:
```

```
        return 1
```

```
    else:
```

```
        return n * fact(n-1)
```

```

if __name__ == "__main__":
    """
    Part 1
    """

    # 1. Define the range for n (starting above 0 as log(0) is undefined)
    n = np.linspace(1, 10, 400)
    n_f = np.linspace(1, 10, 10)
    n_fact = [fact(n) for n in n_f]

    plt.figure(figsize=(10, 6))

    # 2. Calculate f(n) = log(n). np.log() is the natural log (base e)
    f_n = np.ones_like(n)
    plt.plot(n, f_n, label='f(n) = constant')
    f_n = n
    plt.plot(n, f_n, label='f(n) = n')
    f_n = np.log(n)
    plt.plot(n, f_n, label='f(n) = logn')
    f_n = n * np.log(n)
    plt.plot(n, f_n, label='f(n) = nlogn')
    f_n = n ** 2
    plt.plot(n, f_n, label='f(n) = n^2')
    f_n = n ** 3
    plt.plot(n, f_n, label='f(n) = n^3')
    f_n = 2 ** n
    plt.plot(n, f_n, label='f(n) = 2^n')
    plt.plot(n_f, n_fact, label='f(n) = n!')

    # 3. Create the plot
    plt.legend()
    plt.xlim(2, 8)
    plt.yscale('log', base=2)
    plt.ylim(1, 4096)

    # 4. Add labels and a title
    plt.xlabel('n')
    plt.ylabel('f(n)')
    plt.title('Plot of f(n)')

    # 5. Show the result
    plt.show()

    """

```

Part 2

"""

```
run_time = []
```

```
# Test n = 100
```

```
rand_list = np.random.randint(low=0, high=100, size=100).tolist()
```

```
start_time = time( ) # record the starting time
```

```
a_sort(rand_list) #function call goes here
```

```
end_time = time( ) # record the ending time
```

```
elapsed = end_time - start_time
```

```
run_time.append(elapsed)
```

```
print("[X----]")
```

```
# Test n = 1000
```

```
rand_list = np.random.randint(low=0, high=100, size=1000).tolist()
```

```
start_time = time( ) # record the starting time
```

```
a_sort(rand_list) #function call goes here
```

```
end_time = time( ) # record the ending time
```

```
elapsed = end_time - start_time
```

```
run_time.append(elapsed)
```

```
print("[XX---]")
```

```
# Test n = 3000
```

```
rand_list = np.random.randint(low=0, high=100, size=3000).tolist()
```

```
start_time = time( ) # record the starting time
```

```
a_sort(rand_list) #function call goes here
```

```
end_time = time( ) # record the ending time
```

```
elapsed = end_time - start_time
```

```
run_time.append(elapsed)
```

```
print("[XXX--]")
```

```
# Test n = 7500
```

```
rand_list = np.random.randint(low=0, high=100, size=7500).tolist()
```

```
start_time = time( ) # record the starting time
```

```
a_sort(rand_list) #function call goes here
```

```
end_time = time( ) # record the ending time
```

```
elapsed = end_time - start_time
```

```
run_time.append(elapsed)
```

```
print("[XXXX-]")
```

```
# Test n = 10000
```

```
rand_list = np.random.randint(low=0, high=100, size=10000).tolist()
```

```
start_time = time( ) # record the starting time
```

```
a_sort(rand_list) #function call goes here
```

```
end_time = time( ) # record the ending time
```

```
elapsed = end_time - start_time
run_time.append(elapsed)
print("[XXXXX]")

print(f'The run times to be plotted: {run_time}')
# 1. Define the range for n (starting above 0 as log(0) is undefined)
n = np.linspace(1, 10000, 10000)
n2 = [100, 1000, 3000, 7500, 10000]

# 2. Calculate f(n) = log(n). np.log() is the natural log (base e)
f_n = n ** 2
plt.plot(n, f_n, label='O(n²)')
plt.scatter(n2, run_time, label='run time')

# 3. Create the plot
plt.legend()
plt.xscale('log')
plt.yscale('log')

# 4. Add labels and a title
plt.xlabel('n')
plt.ylabel('time')
plt.title('a_sort() Run Times')

# 5. Show the result
plt.show()
```